



Taibah University
College of Computer Science and Engineering
Department of Computer Science

المملكة العربية السعودية
وزارة التعليم العالي
جامعة طيبة
كلية علوم وهندسة الحاسبات

AI371– Cognitive Robotics
AI-Based Recycling Assistant

By Group:

Aram Mohammad Alhejaili (ID: 4451279)

Rand Hamad Alharbi (ID: 4455847)

Lamees Hamad Alrehaili (ID: 4454248)

Section: F2

Supervised By:

Dr. Shurook Almohamade

1. Introduction & Background

Our project aims to develop an intelligent waste recycling system using artificial intelligence and robotics technologies. Manual waste sorting is often a time-consuming process and may lead to significant human errors. Therefore, intelligent automated systems have become an effective solution for improving recycling efficiency, reducing human effort, and increasing sorting accuracy.

The proposed system relies on a robotic arm that detects different types of recyclable waste, such as plastic, cardboard, and metal, and automatically sorts them into the appropriate recycling bins. The system was developed using the Webots simulation environment to simulate a realistic industrial recycling process that includes a conveyor belt and a robotic arm.

The project also utilizes several technologies, including computer vision for image feature extraction, machine learning and deep learning models for waste classification, and reinforcement learning to improve the robot's decision-making during the sorting process. Different classification models were evaluated and compared in order to select the most effective model for the final system.

This project aims to demonstrate how artificial intelligence and robotics can contribute to improving recycling systems and making them more efficient and accurate.

2. The Cognitive Model

2.1. Perception:

Perception is the primary stage where the robot gathers information from its environment. In this project, perception acts as the "eyes" of the system, transforming physical presence into digital data that the AI can process. This layer is composed of two essential sensory inputs:

2.1.1. Object Detection (Distance Sensor):

A distance sensor is mounted directly on the UR5e gripper to act as a proximity trigger. When an item passes under the sensor and the value drops below the threshold (< 450), it signals the system that an object is ready. This localized detection ensures the robot only reacts when necessary, effectively saving computational and energy resources.

2.1.2. Visual Capture (Camera Sensor):

The onboard camera, integrated into the gripper, captures high-quality top-down frames with a consistent perspective and focal length. Once an object is detected, the system applies a bounding box to localize it within the field of view, ensuring precise spatial awareness. This visual data is then converted into a numerical array for accurate real-time classification.



Figure 2.1.2.1 Real-time object detection with bounding box localization

2.2. Learning:

In this project, we implement a hybrid learning approach to enable the robot to classify waste materials and learn the optimal sorting strategy. This is divided into two main phases:

2.2.1. Supervised Learning (Object Recognition):

This phase focuses on the Classification task, where the robot identifies the type of material using a Convolutional Neural Network (CNN). This process is categorized as Supervised Learning because the model was trained on a labeled dataset.

- **Model Architecture:** The system utilizes a pre-trained CNN model (developed using TensorFlow/Keras) that has been fine-tuned to recognize three classes: Cardboard, Metal, and Plastic.
- **Image Preprocessing:** Raw camera images are converted from BGRA to RGB and resized to 224x224 pixels using the OpenCV library to ensure compatibility with the model's input layer.
- **Inference & Output:** The model analyzes the visual features (shapes, textures, and colors) and outputs a probability score. The class with the highest probability is selected as the "State," which is then passed to the decision-making layer.

```
141 #Perception Phase
142 if val < 450:
143     img = camera.getImage()
144     if img:
145         raw_img = np.frombuffer(img, np.uint8).reshape((camera.getHeight(), camera.getWidth(), 4))
146         res = cv2.resize(cv2.cvtColor(raw_img, cv2.COLOR_BGRA2RGB), (224, 224))
147
148         #model prediction
149         predictions = model.predict(np.expand_dims(res, axis=0), verbose=0)[0]
150         state_index = np.argmax(predictions)
151         label = class_names[state_index]
152
153         print(f"Detected: {label} | Picking up...")
154
```

Figure 2.2.1.1 Implementation code of CNN model inference and waste classification

2.2.2. Reinforcement Learning (Decision Making):

To achieve cognitive autonomy, the robot employs Reinforcement Learning (RL) via the Q-Learning algorithm. This allows the robot to learn the correct sorting bin for each material through trial and error. The RL framework in this system is defined by the following core components:

State (s): Defined by the categorical output of the CNN model (e.g., $s \in \{\text{Cardboard, Metal, Plastic}\}$).

Action (a): The movement of the arm to one of the three bins ($a \in \{\text{Bin 0, Bin 1, Bin 2}\}$).

Reward (r): A numerical feedback signal where the agent receives (+1.0 for a successful) sort and (-1.0 as a penalty) for an incorrect action.

- **The Learning Mechanism: Q-Update Equation**

The agent maintains a Q-Table to store the "quality" of each action in each state. After every attempt, the knowledge is updated using the formula [5]:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \cdot \left[r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a) \right]$$

Where:

α (Learning Rate): Set to 0.6, determining how fast the agent adopts new information.

γ (Discount Factor): Set to 0.9, balancing immediate and future rewards.

```
173 #Learning and Rewards
174 actual_correct_bin = CORRECT_BIN_INDEX[label]
175 if chosen_bin == actual_correct_bin:
176     reward = 1.0
177     success_count += 1
178     print(f"Reward: +1")
179
180 else:
181     reward = -1.0
182     print(f"Penalty: -1")
183
184
185
186
187
188
189
190
191 #update q-table file
192 old_value = q_table[state_index, chosen_bin]
193 q_table[state_index, chosen_bin] = old_value + ALPHA * (reward + GAMMA * np.max(q_table[state_index]) - old_value)
194 save_q_table()
195
```

Figure 2.2.2.1 Implementation code of the Q-learning update logic and reward system.

- **Greedy Learning Flow**

The robot follows a strictly greedy approach ($\epsilon = 0$), starting from an empty knowledge base:

Initial Trial: Since the Q-Table starts with all zeros, the robot performs its initial actions to trigger environmental feedback.

Knowledge Acquisition: Once the first rewards (+1.0 or -1.0) are recorded, the table is updated.

Immediate Exploitation: The robot then immediately switches to following the highest-value action (Greedy Action) based on these updates, ensuring it never repeats an error once the correct bin is identified.

```
123
124 #Decision Making
125 def choose_action(state_index):
126     return np.argmax(q_table[state_index])
127
```

Figure 2.2.2.2 Implementation code of the Greedy Action selection mechanism

2.3. Action:

Once the object is identified by the CNN and the optimal bin is selected by the Reinforcement Learning model, the robot executes a series of coordinated physical actions. This section describes the mechanical transition from perception to execution [6].

2.3.1. Motion Control and Kinematics:

The UR5e robotic arm translates AI decisions into physical movements using a position-based control scheme across its six degrees of freedom:

- **Velocity Regulation:** All joints are set to a maximum velocity of 3.14 rad/s. This ensures a fast cycle time while maintaining simulation stability.
- **Coordinate System:** The robot operates based on a set of Pre-defined Joint Angles corresponding to the 3-D coordinates of the conveyor belt (origin) and the three waste bins.

```
89 joint_names = ['shoulder_pan_joint', 'shoulder_lift_joint', 'elbow_joint', 'wrist_1_joint', 'wrist_2_joint', 'wrist_3_joint']
90 joints = [robot.getDevice(n) for n in joint_names]
91
92 #robot's arm velocity
93 def set_arm_velocity(v):
94     for j in joints:
95         if j: j.setVelocity(v)
```

Figure 2.3.1.1 Implementation code for initializing the UR5e joints and defining the global velocity control function.

2.3.2. Sequential Execution (The Sorting Cycle):

To ensure a collision-free and reliable sorting process, the "Action" is executed in four distinct phases:

1) Detection & Grasping:

The robot monitors the environment via a distance sensor. When an item is detected within the threshold, the gripper is activated to secure the object at the pick-up point.

2) Safety Lift & Transfer:

Immediately after grasping, the arm performs a vertical lift to clear the conveyor structure. It then rotates the base and adjusts the joints toward the target bin identified by the Q-learning model.

3) Release:

Upon reaching the target bin's coordinates, the gripper is commanded to release the object. The item is deposited using gravitational force into the assigned bin (Plastic, Metal, or Cardboard).

4) Homing & Reset:

To prepare for the next cycle, the arm executes a return trajectory to its origin state. The simulation then triggers the generation of a new waste object on the belt, completing the autonomous loop.

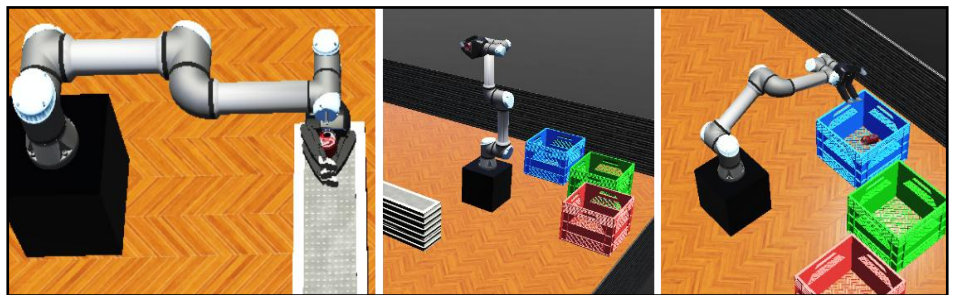


Figure 2.3.2.1 The sequential execution of the autonomous sorting cycle: (1) Grasping, (2) Safety Lifting, (3) Release.

3. System Architecture

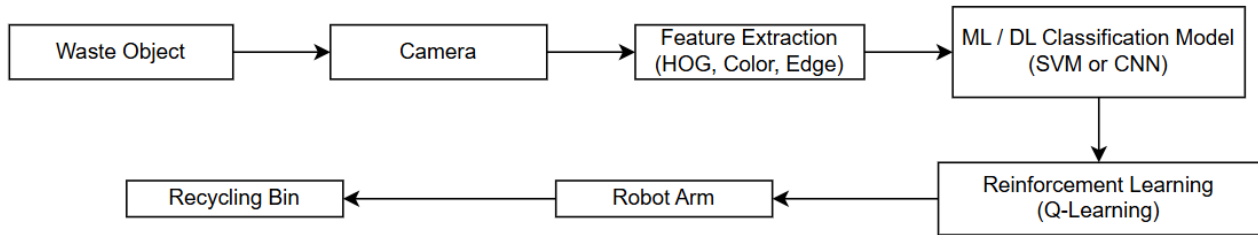


Figure 3.1 System architecture of the intelligent robotic recycling system.

4. Evaluation & Results

4.1. Accuracy

Two classification models were evaluated and compared to determine the most effective approach for recyclable waste classification. A traditional Machine Learning model based on SVM and a Deep Learning CNN model were tested using several evaluation metrics.

The comparison results showed that the CNN model achieved better overall classification performance due to its ability to automatically learn complex image features. The CNN model achieved a test accuracy of 86.22% with a test loss of 0.423, while the SVM model achieved an accuracy of 82.63%.

Therefore, the deep learning CNN model was selected for integration into the final robotic recycling system. The accuracy and loss curves also showed stable training performance and good model generalization.

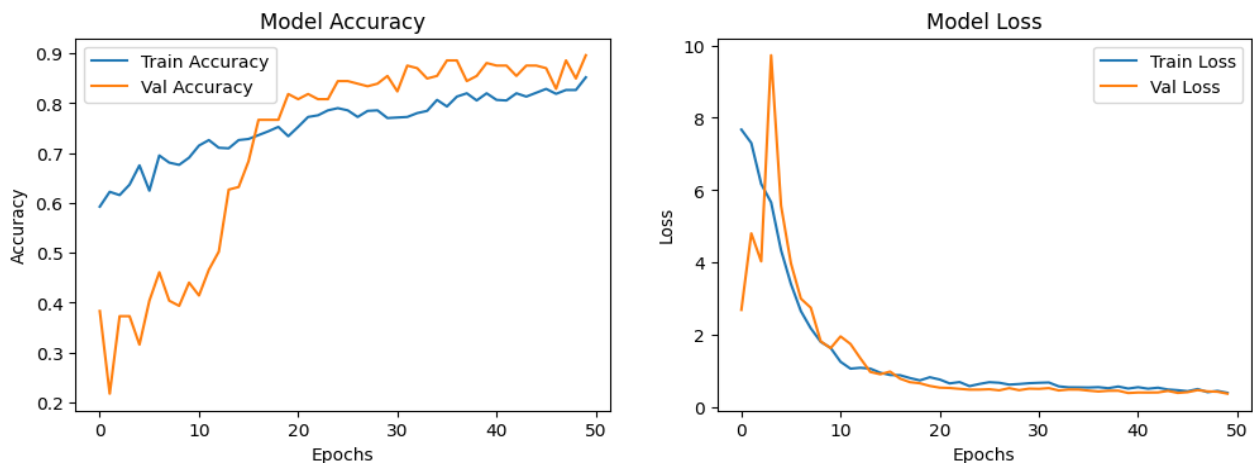


Figure 4.1.1 CNN model training and validation performance: (1)The Model Accuracy Curve, (2) The Model Loss Curve.

4.2. Classification Metrics

For CNN Model:

Material	Precision	Recall	F1-score
Cardboard	0.95	0.85	0.90
Metal	0.79	0.87	0.83
Plastic	0.86	0.86	0.86

For SVM Model:

Material	Precision	Recall	F1-score
Cardboard	0.86	0.80	0.83
Metal	0.81	0.82	0.81
Plastic	0.82	0.85	0.84

4.3. Reinforcement Learning Evaluation

The reinforcement learning component was evaluated inside the Webots simulation environment using Success Rate and Total Reward metrics. The learning curve demonstrated continuous improvement in the robot’s decision-making performance over multiple sorting attempts.

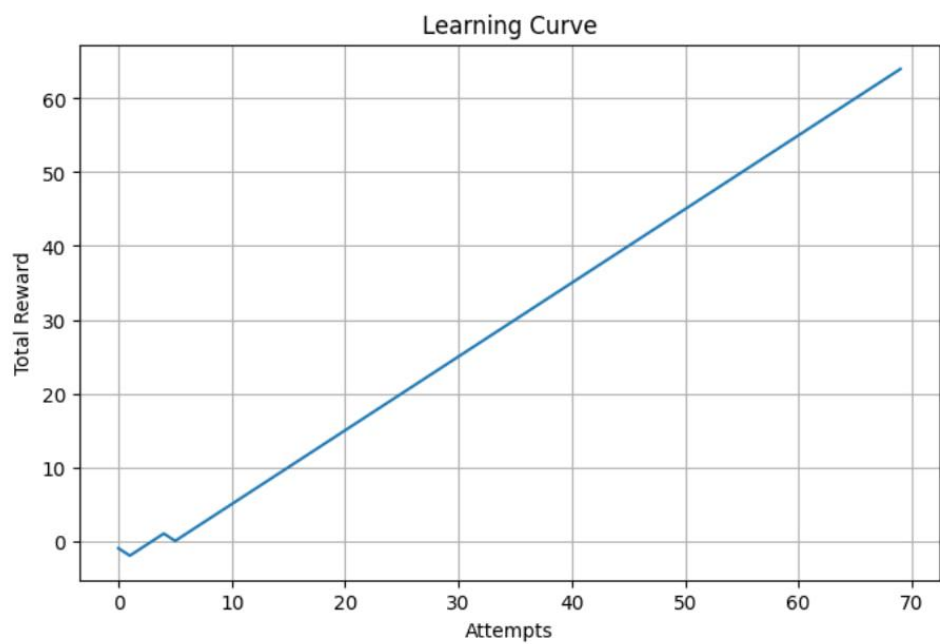


Figure 4.3.1 Reinforcement learning performance across sorting attempts.

5. Challenges and limitations

The project had several challenges during training and implementation inside the Webots environment. One of the main challenges was due to the lack of available objects, we had to import 3D objects from outside [7], and it was difficult to adjust their properties appropriately, which led to similarity between some materials, such as metal and plastic, which caused a few classification errors.

In addition, lighting conditions inside the simulation environment sometimes affected object recognition accuracy. The model also required more training time and improvement using Data Augmentation techniques.

One limitation of the project is that the system was tested only in a simulation environment, so the results may be different in a real-world environment with more complex conditions.

6. Conclusion & Future Improvements

The project shows an AI-Based Recycling Assistant classification system using a CNN model integrated with the Webots simulation environment, in addition to using an SVM model for classification comparison. The system was designed to classify recyclable materials into three categories: cardboard, metal, and plastic.

The results showed that the CNN model achieved good classification and sorting performance within the simulation environment and achieved higher accuracy compared to the SVM model in the test results.

Using Webots helped simulate a realistic robotic recycling environment and demonstrated the potential of using artificial intelligence and robotics to develop efficient recycling systems.

In the future, the project can be improved by increasing the amount of training data and adding more recycling material categories to improve the model accuracy and classification performance.

The system can also be improved by using a more advanced CNN architecture and better training techniques. Future work may also include using more advanced deep learning models to achieve higher accuracy and better performance.

References

- [1] G. Thung and M. Yang, "TrashNet Dataset," GitHub repository. [Online]. Available: <https://github.com/garythung/trashnet> [Accessed: April 2026].
- [2] TensorFlow, "Convolutional Neural Network (CNN) Tutorial," TensorFlow Tutorials. [Online]. Available: <https://www.tensorflow.org/tutorials/images/cnn> [Accessed: May 2026].
- [3] Scikit-learn Developers, "Scikit-learn: Machine Learning in Python," [Online]. Available: <https://scikit-learn.org/> [Accessed: May 2026].
- [4] R. S. Sutton and A. G. Barto, "Reinforcement Learning: An Introduction," MIT Press, 2018.
- [5] "Q-Learning," Reinforcement Learning Path. [Online]. Available: <https://www.reinforcementlearningpath.com/q-learning> [Accessed: May 2026].
- [6] "github," [Online]. Available: https://github.com/DrakerDG/Webots_DG/tree/master/Universal%20Robot%20V3%20Python [Accessed: April 2026].
- [7] CGTrader, "Free 3D Models," CGTrader. [Online]. Available: <https://www.cgtrader.com/free-3d-models>. [Accessed: April, 2026].